Lab 8

LabSection.java

```java
public class LabSection {

    private int numOfReg;

    public LabSection(int r) {
        numOfReg = r;
    }

    public synchronized void register() {
        int updNoOfReg = numOfReg + 1;
        System.out.println("Register-request, registered students: " + updNoOfReg);
        numOfReg = updNoOfReg;
    }

    public synchronized void unregister() {
        int updNoOfReg = numOfReg - 1;
        System.out.println("Unregister-request, registered students: " + updNoOfReg);
        numOfReg = updNoOfReg;
    }

    public double numOfRegisteredStudents() {
        return numOfReg;
    }
}
```

RegisterRunnable.java

```java
public class RegisterRunnable implements Runnable {

    private static final int DELAY = 200;
    private LabSection osLabGroup;

    public RegisterRunnable(LabSection osLab) {
        osLabGroup = osLab;
    }

    public void run() {
        try {
            osLabGroup.register();
            Thread.sleep(DELAY);
        } catch (InterruptedException exception) {
        }
    }
}
```

Misk Aljumah / 451203379

Lab 8

UnregisterRunnable.java

```java
public class UnregisterRunnable implements Runnable {

    private static final int DELAY = 200;
    private LabSection osLabGroup;

    public UnregisterRunnable(LabSection osLab) {
        osLabGroup = osLab;
    }

    public void run() {
        try {
            osLabGroup.unregister();
            Thread.sleep(DELAY);
        } catch (InterruptedException exception) {
        }
    }
}
```

progDemo.java

```java
public class progDemo {

    public static void main(String[] args) {

        LabSection osLab = new LabSection(25);

        for (int i = 1; i <= 25; i++) {

            RegisterRunnable reRun = new RegisterRunnable(osLab);
            UnregisterRunnable unRegRun = new UnregisterRunnable(osLab);

            Thread regThrd = new Thread(reRun);
            Thread unRegThrd = new Thread(unRegRun);

            regThrd.start();
            unRegThrd.start();
        }

        try {
            Thread.sleep(3000);
        } catch (InterruptedException exception) {
        }

        System.out.println("Final number of registered students: "
                + osLab.numOfRegisteredStudents());
    }
}
```

Lab 8

The OutPut Before Fixed :

```
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Final number of registered students: 25.0
```

.Race condition happens when multiple threads access and update the same shared variable at the same time
.In this exercise, the shared variable is numOfReg
.The register thread increases numOfReg by 1, while the unregister thread decreases numOfReg by 1
.If two threads read the same old value before one of them writes the updated value, one update may be lost
.This can lead to an incorrect final number of registered students

The race condition occurs in the register() and unregister() methods .because both methods access and update the shared variable numOfReg

```
int updNoOfReg = numOfReg + 1;
numOfReg = updNoOfReg;

int updNoOfReg = numOfReg - 1;
numOfReg = updNoOfReg;
```

Misk Aljumah / 451203379

Lab 8

The solution is to use the synchronized keyword with the .register() and unregister() methods

This ensures that only one thread can execute one of these .methods at a time

As a result, the shared variable numOfReg is updated safely .and the race condition is avoided

LabSection.java

```java
public class LabSection {

    private int numOfReg;

    public LabSection(int r) {
        numOfReg = r;
    }

    public synchronized void register() {
        int updNoOfReg = numOfReg + 1;
        System.out.println("Register-request, registered students: " + updNoOfReg);
        numOfReg = updNoOfReg;
    }

    public synchronized void unregister() {
        int updNoOfReg = numOfReg - 1;
        System.out.println("Unregister-request, registered students: " + updNoOfReg);
        numOfReg = updNoOfReg;
    }

    public double numOfRegisteredStudents() {
        return numOfReg;
    }
}
```

Misk Aljumah / 451203379

Lab 8

The OutPut After Fixed :

```
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Register-request, registered students: 26
Unregister-request, registered students: 25
Final number of registered students: 25.0
```

Misk Aljumah / 451203379

Lab 8

BankAccount.java

```java
public class BankAccount {

    private double balance;

    public BankAccount() {
        balance = 0;
    }

    public void deposit(double amount) {
        double newBalance = balance + amount;
        System.out.println("Depositing " + amount + ", new balance is: " + newBalance);
        balance = newBalance;
    }

    public void withdraw(double amount) {
        double newBalance = balance - amount;
        System.out.println("Withdrawing " + amount + ", new balance is: " + newBalance);
        balance = newBalance;
    }

    public double getBalance() {
        return balance;
    }
}

```

DepositRunnable.java

```java
public class DepositRunnable implements Runnable {

    private static final int DELAY = 200;
    private BankAccount account;
    private double amount;

    public DepositRunnable(BankAccount anAccount, double anAmount) {
        account = anAccount;
        amount = anAmount;
    }

    public void run() {
        try {
            account.deposit(amount);
            Thread.sleep(DELAY);
        } catch (InterruptedException exception) {
        }
    }
}
```

Lab 8

WithdrawRunnable.java

```java
public class WithdrawRunnable implements Runnable {

    private static final int DELAY = 200;
    private BankAccount account;
    private double amount;

    public WithdrawRunnable(BankAccount anAccount, double anAmount) {
        account = anAccount;
        amount = anAmount;
    }

    public void run() {
        try {
            account.withdraw(amount);
            Thread.sleep(DELAY);
        } catch (InterruptedException exception) {
        }
    }
}
```

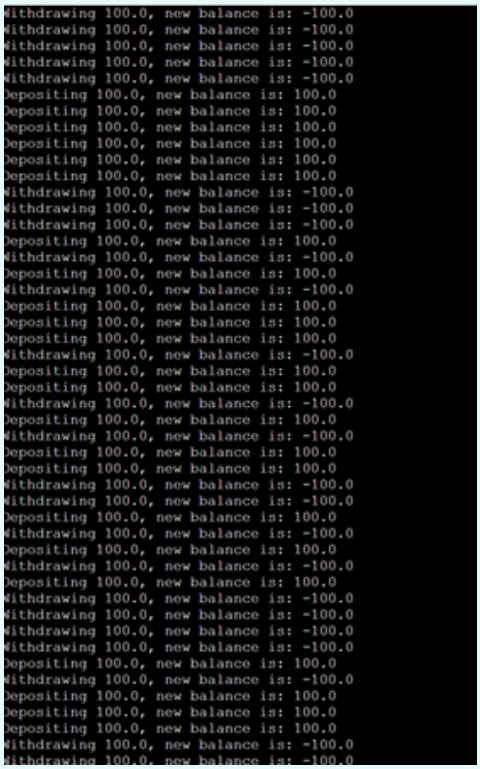
BankAccountThreadRunner.java

```java
public class BankAccountThreadRunner {

    public static void main(String[] args) {

        BankAccount account = new BankAccount();

        final double AMOUNT = 100;
        final int THREADS = 25;

        for (int i = 1; i <= THREADS; i++) {

            DepositRunnable d = new DepositRunnable(account, AMOUNT);
            WithdrawRunnable w = new WithdrawRunnable(account, AMOUNT);

            Thread dt = new Thread(d);
            Thread wt = new Thread(w);

            dt.start();
            wt.start();
        }
    }
}
```

Misk Aljumah / 451203379

Lab 8

The OutPut Before Fixed :



Race condition happens when multiple threads access and update the same variable at the same time.
."In this program, both deposit and withdraw threads access the shared variable "balance
.Each thread reads the balance, modifies it, and writes it back
.If two threads read the same value before updating it, one update may overwrite the other
.This leads to incorrect and inconsistent results

The race condition occurs in the deposit() and withdraw() methods when updateing the balance variable

```
double newBalance = balance + amount;

balance = newBalance;
```

```
double newBalance = balance - amount;

balance = newBalance;
```

Misk Aljumah / 451203379

Lab 8

The solution is to use the synchronized keyword

This ensures that only one thread can execute the deposit or withdraw method at a time

This prevents multiple threads from accessing and modifying the shared variable simultaneously and avoids race condition

BankAccount.java

```java
public class BankAccount {

    private double balance;

    public BankAccount() {
        balance = 0;
    }

    public synchronized void deposit(double amount) {
        double newBalance = balance + amount;
        System.out.println("Depositing " + amount + ", new balance is: " + newBalance);
        balance = newBalance;
    }

    public synchronized void withdraw(double amount) {
        double newBalance = balance - amount;
        System.out.println("Withdrawing " + amount + ", new balance is: " + newBalance);
        balance = newBalance;
    }

    public double getBalance() {
        return balance;
    }
}
```

Lab 8

The OutPut After Fixed :

```
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Depositing 100.0, new balance is: 200.0
Withdrawing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Final Balance: 0.0
```